Appendix B Kubernetes in production with DigitalOcean

This appendix covers

- Running a Kubernetes cluster on DigitalOcean
- Running a PostgreSQL database on DigitalOcean
- Running Redis on DigitalOcean
- Running RabbitMQ using a Kubernetes Operator
- Running Keycloak using a Helm chart

Kubernetes is the de facto standard for deploying and managing containerized workloads. We've been relying on a local Kubernetes cluster to deploy applications and services in the Polar Bookshop system throughout the book. For production, we need something else.

All major cloud providers offer a managed Kubernetes service. In this appendix, you'll see how to use DigitalOcean to spin up a Kubernetes cluster. We'll also rely on other managed services provided by the platform, including PostgreSQL and Redis. Finally, this appendix will guide you through the deployment of RabbitMQ and Keycloak directly in Kubernetes.

Before moving on, you need to ensure that you have a DigitalOcean account. When you sign up, DigitalOcean offers a 60-day free trial with a $100 credit that is more than enough to go through the examples in chapter 15. Follow the instructions on the official website to create an account and start a free trial (**https://try.digitalocean.com/freetrialoffer**).

NOTE The source code repository accompanying this book contains additional instructions for setting up a Kubernetes cluster on a few different cloud platforms, in case you'd like to use something other than DigitalOcean.

There are two main options for interacting with the DigitalOcean platform. The first one is through the web portal (**https://cloud.digitalocean.com**), which is very convenient

for exploring the available services and their features. The second option is via doctl, the DigitalOcean CLI. That's what we're going to use in the following sections.

You can find instructions for installing doctl on the official website (**https://docs.digitalocean.com/reference/doctl/how-to/install**). If you're on macOS or Linux, you can easily install it using Homebrew:

```
$ brew install doctl
```

You can follow the subsequent instructions on the same doctl page to generate an API token and grant doctl access to your DigitalOcean account.

NOTE In a real production scenario, you would automate the platform management tasks using a tool like Terraform or Crossplane. That is usually the responsibility of the platform team, not of application developers, so I won't add extra complexity here by introducing yet another tool. Instead we'll use the DigitalOcean CLI directly. If you're interested in Terraform, Manning has a book in its catalog on the subject: *Terraform in Action* by Scott Winkler (Manning, 2021; **https://www.manning.com/books/terraform-in-action**). For Crossplane, I recommend reading chapter 4 of *Continuous Delivery for Kubernetes* by Mauricio Salatino (**https://livebook.manning.com/book/continuous-delivery-for-kubernetes/chapter-4**).

B.1 Running a Kubernetes cluster on DigitalOcean

The first resource we need to create on DigitalOcean is a Kubernetes cluster. You could rely on the IaaS capabilities offered by the platform and install a Kubernetes cluster manually on top of virtual machines. Instead, we'll move up the abstraction staircase and go for a solution managed by the platform. When we use DigitalOcean Kubernetes (**https://docs.digitalocean.com/products/kubernetes**), the platform will take care of many infrastructural concerns, so that we developers can focus more on application development.

You can straightforwardly create a new Kubernetes cluster using doctl. I promised that we would deploy Polar Bookshop in a real production environment, and that's what we'll do, although I won't ask you to size and configure the cluster as I would in a real scenario.

For starters, setting up a Kubernetes cluster is not a developer's responsibility—it's a job for the platform team. Second, it would require more in-depth coverage of Kubernetes

than is provided by this book to fully understand the configuration. Third, I don't want you to incur extra costs on DigitalOcean for using a lot of computational resources and services. Cost optimization is a cloud property that applies to real applications. However, it might become expensive if you're trying things out or running demo applications. Please keep an eye on your DigitalOcean account to monitor when your free trial and $100 credit expire.

Each cloud resource can be created in a data center hosted in a specific geographical region. For better performance, I recommend you choose one near to you. I'll use "Amsterdam 3" (ams3), but you can get the complete list of regions with the following command:

```
$ doctl k8s options regions
```

Let's go ahead and initialize a Kubernetes cluster using DigitalOcean Kubernetes (DOKS). It will be composed of three worker nodes, for which you can decide the technical specifications. You can choose between different options in terms of CPU, memory, and architecture. I'll use nodes with 2 vCPU and 4 GB of memory:

```
$ doctl k8s cluster create polar-cluster \                          (1)
  --node-pool "name=basicnp;size=s-2vcpu-4gb;count=3;label=type=basic;" \ (2)
  --region <your_region>                                            (3)
```

(1) Defines the name of the cluster to create

(2) Provides the requested specifications for the worker nodes

(3) The data center region of your choice, such as "ams3"

NOTE If you'd like to know more about the different compute options and their prices, you can use the doctl compute size list command.

The cluster provisioning will take a few minutes. In the end, it will print out the unique ID assigned to the cluster. Take note, since you'll need it later. You can fetch the cluster ID at any time by running the following command (I have filtered the results for the sake of clarity):

```
$ doctl k8s cluster list
```

```
ID          Name          Region    Status    Node Pools
<cluster-id>    polar-cluster    ams3     running    basicnp
```

At the end of the cluster provisioning, doctl will also configure the context for your Kubernetes CLI so that you can interact with the cluster running on DigitalOcean from your computer, similar to what you've done so far with your local cluster. You can verify the current context for kubectl by running the following command:

```
$ kubectl config current-context
```

NOTE If you want to change the context, you can run kubectl config use-context <context-name>.

Once the cluster is provisioned, you can get information about the worker nodes as follows:

```
$ kubectl get nodes

NAME     STATUS   ROLES    AGE    VERSION
<node-1>   Ready    <none>   2m34s   v1.24.3
<node-2>   Ready    <none>   2m36s   v1.24.3
<node-3>   Ready    <none>   2m26s   v1.24.3
```

Do you remember the Octant dashboard you used to visualize the workloads on your local Kubernetes cluster? You can now use it to get information about the cluster on DigitalOcean as well. Open a Terminal window and start Octant with the following command:

```
$ octant
```

Octant will open in your browser and show data from your current Kubernetes context, which should be the cluster on DigitalOcean. From the upper-right menu, you can switch between contexts from the drop-down box, as shown in figure B.1.

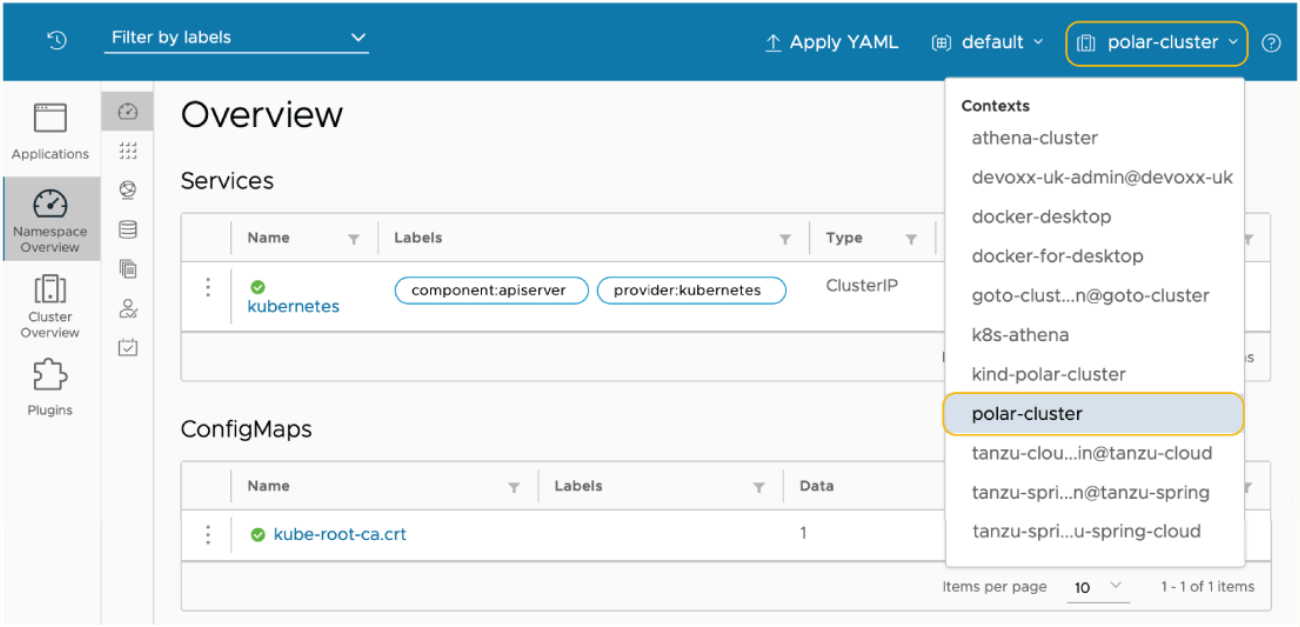


Figure B.1 Octant lets you visualize workloads from different Kubernetes clusters by switching contexts.

As I mentioned in chapter 9, Kubernetes doesn't come packaged with an Ingress Controller; it's up to you to install one. Since we'll rely on an Ingress resource to allow traffic from the public internet to the cluster, we need to install an Ingress Controller. Let's install the same one we used locally: ingress-nginx.

In your polar-deployment repository, create a new kubernetes/platform/production folder, and copy over the content from the Chapter15/15-end/polar-deployment/kubernetes/platform/production folder in the source code repository accompanying the book.

Then open a Terminal window, navigate to the kubernetes/platform/production/ingress-nginx folder in your polar-deployment project, and run the following command to deploy ingress-nginx to your production Kubernetes cluster:

```
$ ./deploy.sh
```

Feel free to open the file and look at the instructions before running it.

NOTE You might need to make the script executable first with the command chmod +x deploy.sh.

In the next section, you'll see how to initialize a PostgreSQL database on DigitalOcean.

B.2 Running a PostgreSQL database on DigitalOcean

In most of the book, you've been running PostgreSQL database instances as containers, both in Docker and in your local Kubernetes cluster. In production, we'd like to take advantage of the platform and use a managed PostgreSQL service provided by DigitalOcean (**https://docs.digitalocean.com/products/databases/postgresql**).

The applications we developed throughout the book are cloud native and follow the 15-Factor methodology. As such, they treat backing services as attached resources that can be swapped without changing anything in the application code. Furthermore, we followed the environment parity principle and used a real PostgreSQL database both for development and testing, and it's the same database we want to use in production.

Moving from a PostgreSQL container running in your local environment to a managed service with high availability, scalability, and resilience is a matter of changing the values of a few configuration properties for Spring Boot. How great is that?

First, create a new PostgreSQL server named polar-postgres, as shown in the following code snippet. We'll use PostgreSQL 14, which is the same version we used for development and testing. Remember to replace <your_region> with the geographical region you'd like to use. It should be the same as the region you used for the Kubernetes cluster. In my case, it's ams3:

```
$ doctl databases create polar-db \
   --engine pg \
   --region <your_region> \
   --version 14
```

The database server provisioning will take several minutes. You can verify the installation status with the following command (I have filtered the result for the sake of clarity):

```
$ doctl databases list

ID           Name      Engine   Version   Region   Status
<polar-db-id>   polar-db   pg      14       ams3     online
```

When the database is online, your database server is ready. Take note of the database server ID. You'll need it later.

To mitigate unnecessary attack vectors, you can configure a firewall so that the PostgreSQL server is only accessible from the Kubernetes cluster created previously. Remember that I asked you to take notes of the resource IDs for PostgreSQL and Kubernetes? Use them in the following command to configure the firewall and secure access to the database server:

```
$ doctl databases firewalls append <postgres_id> --rule k8s:<cluster_id>
```

Next, let's create two databases to be used by Catalog Service (polardb_catalog) and Order Service (polardb_order). Remember to replace <postgres_id> with your PostgreSQL resource ID:

```
$ doctl databases db create <postgres_id> polardb_catalog
$ doctl databases db create <postgres_id> polardb_order
```

Finally, let's retrieve the details for connecting to PostgreSQL. Remember to replace <postgres_id> with your PostgreSQL resource ID:

```
$ doctl databases connection <postgres_id> --format Host,Port,User,Password

Host        Port        User        Password
<db-host>   <db-port>   <db-user>   <db-password>
```

Before concluding this section, let's create some Secrets in the Kubernetes cluster with the PostgreSQL credentials required by the two applications. In a real-world scenario, we should create dedicated users for the two applications and grant limited privileges. For simplicity, we'll use the admin account for both.

First, create a Secret for Catalog Service using the information returned by the previous doctl command:

```
$ kubectl create secret generic polar-postgres-catalog-credentials \
    --from-literal=spring.datasource.url=
➥jdbc:postgresql://<postgres_host>:<postgres_port>/polardb_catalog \
    --from-literal=spring.datasource.username=<postgres_username> \
    --from-literal=spring.datasource.password=<postgres_password>
```

Similarly, create a Secret for Order Service. Pay attention to the slightly different syntax required by Spring Data R2DBC for the URL:

```
$ kubectl create secret generic polar-postgres-order-credentials \
    --from-literal="spring.flyway.url=
➥jdbc:postgresql://<postgres_host>:<postgres_port>/polardb_order" \
    --from-literal="spring.r2dbc.url=
➥r2dbc:postgresql://<postgres_host>:<postgres_port>/polardb_order?
➥ssl=true&sslMode=require" \
    --from-literal=spring.r2dbc.username=<postgres_username> \
    --from-literal=spring.r2dbc.password=<postgres_password>
```

That's it for PostgreSQL. In the next section, you'll see how to initialize Redis using DigitalOcean.

B.3 Running Redis on DigitalOcean

In most of the book, you've been running Redis instances as containers, both in Docker and in your local Kubernetes cluster. In production we'd like to take advantage of the platform and use a managed Redis service provided by DigitalOcean (https://docs.digitalocean.com/products/databases/redis/).

Once again, since we followed the 15-Factor methodology, we can swap the Redis backing service used by Edge Service without changing anything in the application code. We'll only need to change a few configuration properties for Spring Boot.

First, create a new Redis server named polar-redis as shown in the following code snippet. We'll use Redis 7, which is the same version we used for development and testing. Remember to replace <your_region> with the geographical region you'd like to use. It should be the same region you used for the Kubernetes cluster. In my case, it's ams3:

```
$ doctl databases create polar-redis \
    --engine redis \
    --region <your_region> \
    --version 7
```

The Redis server provisioning will take several minutes. You can verify the installation status with the following command (I have filtered the result for the sake of clarity):

```
$ doctl databases list

ID          Name        Engine   Version   Region   Status
<redis-db-id>   polar-redis   redis   7      ams3     creating
```

When the server is online, your Redis server is ready. Take note of the Redis resource ID. You'll need it later.

To mitigate unnecessary attack vectors, we can configure a firewall so that the Redis server is only accessible from the Kubernetes cluster created previously. Remember that I asked you to take notes of the resource IDs for Redis and Kubernetes? Use them in the following command to configure the firewall and secure access to the Redis server:

```
$ doctl databases firewalls append <redis_id> --rule k8s:<cluster_id>
```

Finally, let's retrieve the details for connecting to Redis. Remember to replace <redis_id> with your Redis resource ID:

```
$ doctl databases connection <redis_id> --format Host,Port,User,Password

Host          Port          User          Password
<redis-host>   <redis-port>   <redis-user>   <redis-password>
```

Before concluding this section, let's create a Secret in the Kubernetes cluster with the Redis credentials required by Edge Service. In a real-world scenario, we should create a dedicated user for the application and grant limited privileges. For simplicity, we'll use the default account. Populate the Secret with the information returned by the previous doctl command:

```
$ kubectl create secret generic polar-redis-credentials \
  --from-literal=spring.redis.host=<redis_host> \
  --from-literal=spring.redis.port=<redis_port> \
  --from-literal=spring.redis.username=<redis_username> \
  --from-literal=spring.redis.password=<redis_password> \
  --from-literal=spring.redis.ssl=true
```

That's it for Redis. The following section will cover how to deploy RabbitMQ using a Kubernetes Operator.

B.4 Running RabbitMQ using a Kubernetes Operator

In the previous sections, we initialized and configured PostgreSQL and Redis servers that are offered and managed by the platform. We can't do the same for RabbitMQ because DigitalOcean doesn't have a RabbitMQ offering, similar to other cloud providers like Azure or GCP.

A popular and convenient way of deploying and managing services like RabbitMQ in a Kubernetes cluster is to use the *operator* pattern. Operators are "software extensions to Kubernetes that make use of custom resources to manage applications and their components" (**https://kubernetes.io/docs/concepts/extend-kubernetes/operator**).

Think about RabbitMQ. To use it in production, you'll need to configure it for high availability and resilience. Depending on the workload, you might want to scale it dynamically. When a new version of the software is available, you'll need a reliable way of upgrading the service and migrating existing constructs and data. You could perform all those tasks manually. Or you could use an Operator to capture all those operational requirements and instruct Kubernetes to take care of them automatically. In practice, an Operator is an application that runs on Kubernetes and interacts with its API to accomplish its functionality.

The RabbitMQ project provides an official Operator to run the event broker on a Kubernetes cluster (**www.rabbitmq.com**). I have already configured all the necessary resources to use the RabbitMQ Kubernetes Operator and prepared a script to deploy it.

Open a Terminal window, go to your Polar Deployment project (polar-deployment), and navigate to the kubernetes/platform/production/rabbitmq folder. You should have copied that folder over to your repository when configuring the Kubernetes cluster. If that's not the case, please do so now from the source code repository accompanying this book (Chapter15/15-end/polar-deployment/platform/production/rabbitmq).

Then run the following command to deploy RabbitMQ to your production Kubernetes cluster:

```
$ ./deploy.sh
```

Feel free to open the file and look at the instructions before running it.

NOTE You might need to make the script executable first with the command chmod +x deploy.sh.

The script will output details about all the operations performed to deploy RabbitMQ. Finally, it will create a polar-rabbitmq-credentials Secret with the credentials that Order Service and Dispatcher Service will need to access RabbitMQ. You can verify that the Secret has been successfully created as follows:

```
$ kubectl get secrets polar-rabbitmq-credentials
```

The RabbitMQ broker is deployed in a dedicated rabbitmq-system namespace. Applications can interact with it at polar-rabbitmq.rabbitmq-system.svc.cluster.local on port 5672.

That's it for RabbitMQ. In the next section, you'll see how to deploy a Keycloak server to a production Kubernetes cluster.

B.5 Running Keycloak using a Helm chart

As with RabbitMQ, DigitalOcean doesn't provide a managed Keycloak service. The Keycloak project is working on an Operator, but it's still in beta at the time of writing, so we'll deploy it using a different approach: Helm charts.

Think of Helm as a package manager. To install software on your computer, you would use one of the operating system package managers, like apt (Ubuntu), Homebrew (macOS), or Chocolatey (Windows). In Kubernetes you can similarly use Helm, but they're called *charts* instead of *packages*.

Go ahead and install Helm on your computer. You can find the instructions on the official website (**https://helm.sh**). If you are on macOS or Linux, you can install Helm with Homebrew:

```
$ brew install helm
```

I have already configured all the necessary resources to use the Keycloak Helm chart provided by Bitnami (**https://bitnami.com**), and I've prepared a script to deploy it. Open a Terminal window, go to your Polar Deployment project (polar-deployment), and navigate to the kubernetes/platform/production/keycloak folder. You should have copied that folder over to your repository when configuring the Kubernetes cluster. If that's not the case, please do so now from the source code repository accompanying this book (Chapter15/15-end/polar-deployment/platform/production/keycloak).

Then run the following command to deploy Keycloak to your production Kubernetes cluster:

```
$ ./deploy.sh
```

Feel free to open the file and look at the instructions before running it.

NOTE You might need to make the script executable first with the command `chmod +x deploy.sh`.

The script will output details about all the operations performed to deploy Keycloak and print the admin username and password you can use to access the Keycloak Admin Console. Feel free to change the password after your first login. Note the credentials down, since you might need them later. The deployment can take several minutes to complete, so it's a good time to take a break and drink a beverage of your choice as a reward for everything you have accomplished so far. Good job!

Finally, the script will create a `polar-keycloak-client-credentials` Secret with the Client secret that Edge Service will need to authenticate with Keycloak. You can verify that the Secret has been successfully created as follows. The value is generated randomly by the script:

```
$ kubectl get secrets polar-keycloak-client-credentials
```

NOTE The Keycloak Helm chart spins up a PostgreSQL instance inside the cluster and uses it to persist the data used by Keycloak. We could have integrated it with the PostgreSQL service managed by DigitalOcean, but the configuration on the Keycloak side would have been quite complicated. If you'd like to use an external PostgreSQL database, you can refer to the Keycloak Helm chart documentation (**https://bitnami.com/stack/keycloak/helm**).

The Keycloak server is deployed in a dedicated `keycloak-system` namespace. Applications can interact with it at `polar-keycloak.keycloak-system.svc.cluster.local` on port `8080` from within the cluster. It's also exposed outside the cluster via a public IP address. You can find the external IP address with the following command:

```
$ kubectl get service polar-keycloak -n keycloak-system

NAME           TYPE           CLUSTER-IP      EXTERNAL-IP
polar-keycloak   LoadBalancer   10.245.191.181   <external-ip>
```

The platform might take a few minutes to provision a load balancer. During the provisioning, the EXTERNAL-IP column will show a <pending> status. Wait and try again until an IP address is shown. Note it down, since we're going to use it in multiple scenarios.

Since Keycloak is exposed via a public load balancer, you can use the external IP address to access the Admin Console. Open a browser window, navigate to http://<external-ip>/admin, and log in with the credentials returned by the previous deployment script.

Now that you have a public DNS name for Keycloak, you can define a couple of Secrets to configure the Keycloak integration in Edge Service (OAuth2 Client), Catalog Service, and Order Service (OAuth2 Resource Servers). Open a Terminal window, navigate to the kubernetes/platform/production/keycloak folder in your polar-deployment project, and run the following command to create the Secrets that the applications will use to integrate with Keycloak. Feel free to open the file and look at the instructions before running it. Remember to replace <external-ip> with the external IP address assigned to your Keycloak server:

```
$ ./create-secrets.sh http://<external-ip>/realms/PolarBookshop
```

That's it for Keycloak. The following section will show you how to deploy Polar UI to the production cluster.

B.6 Running Polar UI

Polar UI is a single-page application built with Angular and served by NGINX. As you saw in chapter 11, I have already prepared a container image you can use to deploy this application, since frontend development is out of scope for this book.

Open a Terminal window, go to your Polar Deployment project (polar-deployment), and navigate to the kubernetes/platform/production/polar-ui folder. You should have copied that folder over to your repository when configuring the Kubernetes cluster. If that's not the case, please do so now from the source code repository accompanying this book (Chapter15/15-end/polar-deployment/platform/production/polar-ui).

Then run the following command to deploy Polar UI to your production Kubernetes cluster. Feel free to open the file and look at the instructions before running it:

```
$ ./deploy.sh
```

NOTE You might need to make the script executable first with the command chmod +x deploy.sh.

Now that you have Polar UI and all the main platform services up and running, you can proceed with reading chapter 15 and complete the configuration of all the Spring Boot applications in Polar Bookshop for production deployment.

B.7 Deleting all cloud resources

When you're done experimenting with the Polar Bookshop project, follow the instructions in this section to delete all the cloud resources created on DigitalOcean. That's fundamental to avoid incurring unexpected costs.

First, delete the Kubernetes cluster:

```
$ doctl k8s cluster delete polar-cluster
```

Next, delete the PostgreSQL and Redis databases. You'll need to know their IDs first, so run this command to extract that information:

```
$ doctl databases list

ID              Name          Engine    Version    Region    Status
<polar-db-id>   polar-db      pg        14         ams3      online
<redis-db-id>   polar-redis   redis     7          ams3      creating
```

Then go ahead and delete both of them using the resource identifiers returned by the previous command:

```
$ doctl databases delete <polar-db-id>
$ doctl databases delete <redis-db-id>
```

Finally, open a browser window, navigate to the DigitalOcean web interface (**https://cloud.digitalocean.com**), and go through the different categories of cloud resources in your account to verify that there's no outstanding services. If there are, delete them. There could be load balancers or persistent volumes created as a side effect of creating a cluster or a database, and that may not have been deleted by the previous commands.

Previous chapter
< Appendix A Setting up your development environment

Next chapter
index >